
**Sunita D. Rathod**

# What is Flask?

- Flask is a web application framework written in Python.
- It is developed by **Armin Ronacher**, who leads an international group of Python enthusiasts named Pocco.
- Flask is based on the Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.

# WSGI

- Web Server Gateway Interface (WSGI) has been adopted as a standard for Python web application development.

- WSGI is a specification for a universal interface between the web server and the web applications.

# Werkzeug

- It is a WSGI toolkit, which implements requests, response objects, and other utility functions.
- This enables building a web framework on top of it.
- The Flask framework uses Werkzeug as one of its bases.

# Jinja2

- Jinja2 is a popular templating engine for Python.
- A web templating system combines a template with a certain data source to render dynamic web pages.

# Flask

- Flask is often referred to as a micro framework.
- It aims to keep the core of an application simple yet extensible.
- Flask does not have built-in abstraction layer for database handling, nor does it have form a validation support.
- Instead, Flask supports the extensions to add such functionality to the application.

# Flask Environment Setup

- Python 2.7 or higher is a pre-requisite to installing flask on your system

- The following command installs **virtualenv**

```
$ pip install virtualenv
```

- Once installed, new virtual environment is created in a folder.

```
mkdir newproj
cd newproj
virtualenv venv
```

# Flask Environment Setup

- To activate corresponding environment, on **Linux/OS X**, use the following −

  $ venv/bin/activate

- On **Windows**, following can be used

  $ venv\scripts\activate

- We are now ready to install Flask in this environment.

  $ pip install Flask

- The above command can be run directly, without virtual environment for system-wide installation.

# First program

- In order to test **Flask** installation, type the following code in the editor as **Hello.py**

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

# Explaination

- Importing flask module in the project is mandatory. An object of Flask class is our **WSGI** application.

- Flask constructor takes the name of **current module (__name__)** as argument.

- The **route()** function of the Flask class is a decorator, which tells the application which URL should call the associated function.

  **Syntax:** app.route(rule, options)

- **Rule -** represents the URL binding with the function.
- **Options -** represent the parameters that are associated with the rule object.

# Explaination

- **‘/’** URL is bound with **hello_world()** function. Hence, when the home page of web server is opened in browser, the output of this function will be rendered.

- Finally the **run()** method of Flask class runs the application on the local development server.

- Syntax: app.run(host, port, debug, options)

| Option | Explanation |
|---|---|
| Host | The default host name is 127.0.0.1 (localhost) |
| Port | Port number on which server is listening default is 5000 |
| Debug | Default is false. Provides debug info is set to true |
| Options | Contains information to be forwarded to server |

# Execution

- The above given **Python** script is executed from Python shell.

  Python Hello.py

- A message in Python shell informs you that

  Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

- Open the above URL **(localhost:5000)** in the browser. **'Hello World'** message will be displayed on it.

# Flask – Routing

- Modern web frameworks use the routing technique to help a user remember application URLs.

- The **route()** decorator in Flask is used to bind URL to a function. For example −

```
@app.route('/hello')
def hello_world():
   return 'hello world'
```

- Here, URL **'/hello'** rule is bound to the **hello_world()** function.

# Flask – Routing

- The **add_url_rule()** function of an application object is also available to bind a URL with a function as in the above example, **route()** is used.

- A decorator's purpose is also served by the following representation −

```
def hello_world():
   return 'hello world'
app.add_url_rule('/', 'hello', hello_world)
```

# Flask – Variable Rules

- It is possible to build a URL dynamically, by adding variable parts to the rule parameter.

- This variable part is marked as **<variable-name>**.

```
from flask import Flask
app = Flask(__name__)

@app.route('/hello/<name>')
def hello_name(name):
   return 'Hello %s!' % name

if __name__ == '__main__':
   app.run(debug = True)
```

**Save and open URL**

**http://localhost:5000/hello/Sunita**

**Output:**

**Hello Sunita**

# Flask – Variable Rules

- In addition to the default string variable part, rules can be constructed using the following converters −

| Sr.No. | Converters & Description |
|---|---|
| 1 | **int**<br>accepts integer |
| 2 | **float**<br>For floating point value |
| 3 | **path**<br>accepts slashes used as directory separator character |

# Flask – Variable Rules

```
from flask import Flask
app = Flask(__name__)

@app.route('/blog/<int:postID>')
def show_blog(postID):
    return 'Blog Number %d' % postID

@app.route('/rev/<float:revNo>')
def revision(revNo):
    return 'Revision Number %f' % revNo

if __name__ == '__main__':
    app.run()
```

# Flask – Variable Rules

- Run the above code from Python Shell. Visit the URL **http://localhost:5000/blog/11** in the browser.

Blog Number 11

- Enter this URL in the browser
  - **http://localhost:5000/rev/1.1**

Revision Number 1.100000

# URL Building

- The **url_for()** function is very useful for dynamically building a URL for a specific function.

- The function accepts the name of a function as first argument, and one or more keyword arguments, each corresponding to the variable part of URL.

# URL Building

```
from flask import Flask, redirect, url_for
app = Flask(__name__)

@app.route('/admin')
def hello_admin():
   return 'Hello Admin'

@app.route('/guest/<guest>')
def hello_guest(guest):
   return 'Hello %s as Guest' % guest

@app.route('/user/<name>')
def hello_user(name):
   if name =='admin':
      return redirect(url_for('hello_admin'))
   else:
      return redirect(url_for('hello_guest',guest = name))

if __name__ == '__main__':
   app.run(debug = True)
```

# URL Building

Save the above code and run from Python shell.

Open the browser and enter URL as
− **http://localhost:5000/user/admin**

The application response in browser is −

**Hello Admin**

Enter the following URL in the browser
− **http://localhost:5000/user/sunita**

The application response now changes to −

**Hello sunita as Guest**

# Flask – HTTP methods

- Http protocol is the foundation of data communication in world wide web.

| Sr.No. | Methods & Description |
|---|---|
| 1 | **GET**<br>Sends data in unencrypted form to the server. Most common method. |
| 2 | **HEAD**<br>Same as GET, but without response body |
| 3 | **POST**<br>Used to send HTML form data to server. Data received by POST method is not cached by server. |
| 4 | **PUT**<br>Replaces all current representations of the target resource with the uploaded content. |
| 5 | **DELETE**<br>Removes all current representations of the target resource given by a URL |

# POST method in URL routing

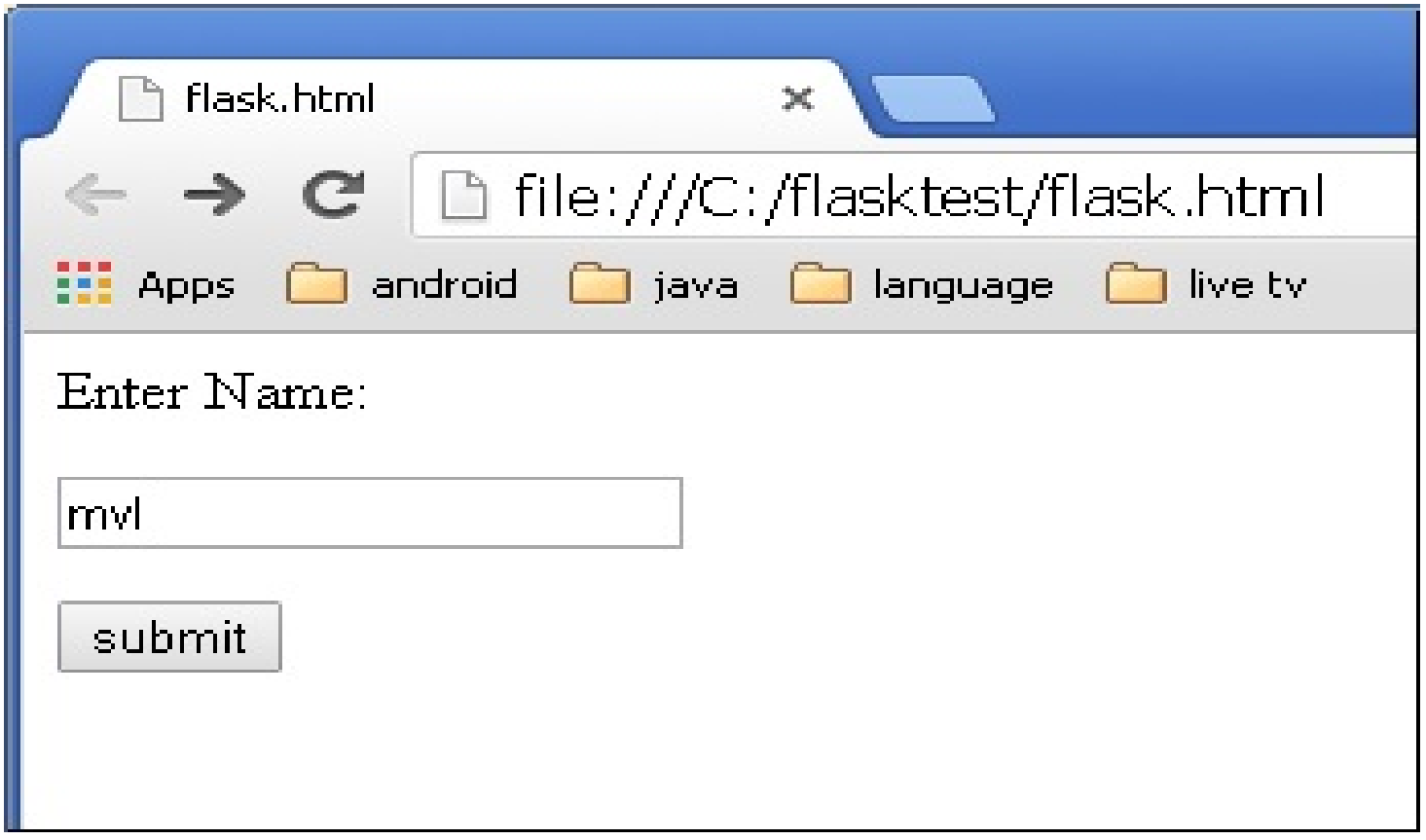
```
<html>
   <body>
      <form action = "http://localhost:5000/login" method = "post">
         <p>Enter Name:</p>
         <p><input type = "text" name = "nm" /></p>
         <p><input type = "submit" value = "submit" /></p>
      </form>
   </body>
</html>
```

Save the following script as login.html

enter the following script in Python shell.

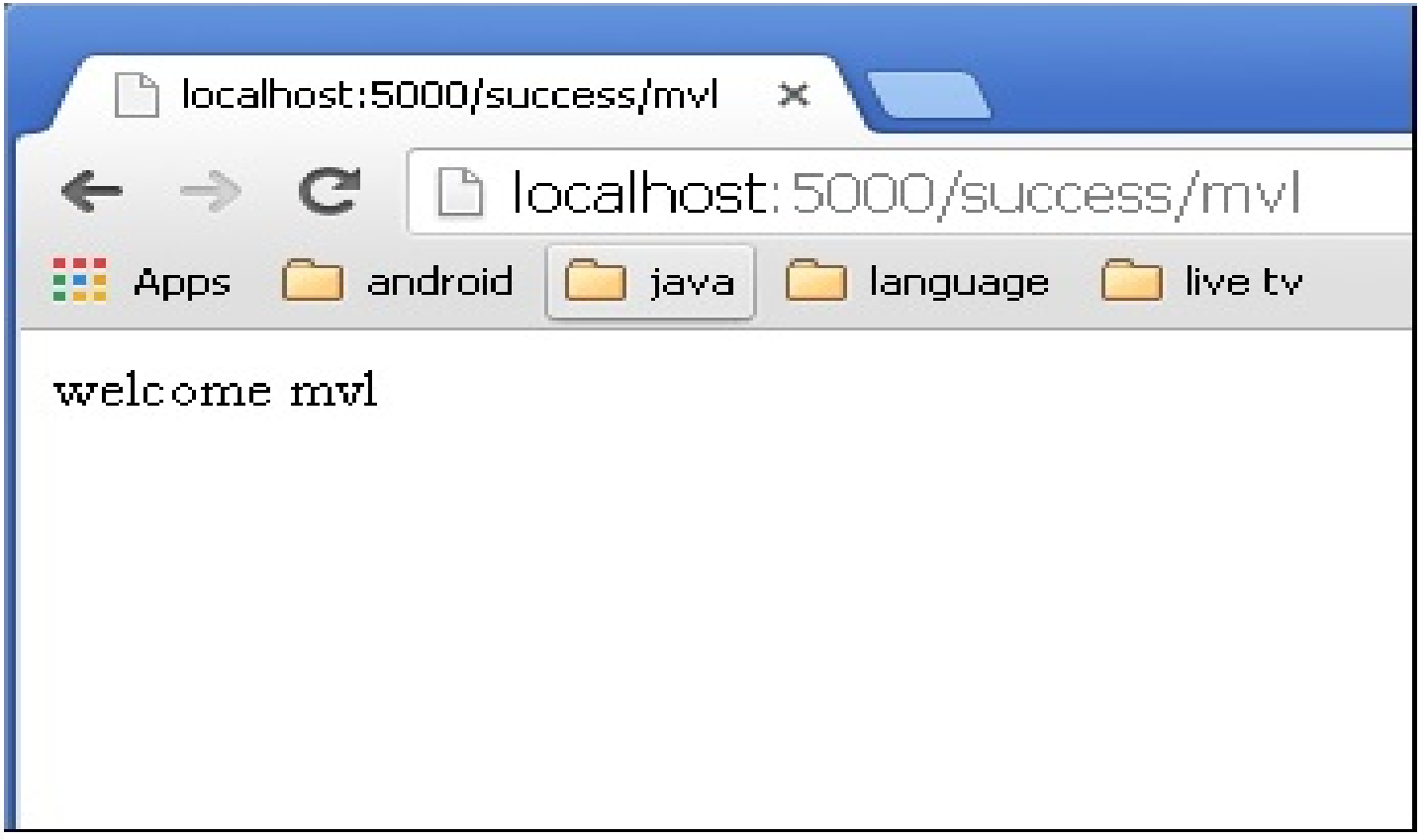
```
from flask import Flask, redirect, url_for, request
app = Flask(__name__)

@app.route('/success/<name>')
def success(name):
   return 'welcome %s' % name

@app.route('/login',methods = ['POST', 'GET'])
def login():
   if request.method == 'POST':
      user = request.form['nm']
      return redirect(url_for('success',name = user))
   else:
      user = request.args.get('nm')
      return redirect(url_for('success',name = user))

if __name__ == '__main__':
   app.run(debug = True)
```

- open **login.html** in the browser, enter name in the text field and click **Submit**.



- Form data is POSTed to the URL in action clause of form tag.
- **http://localhost/login** is mapped to the **login()** function.
- Since the server has received data by **POST** method, value of ‘nm’ parameter obtained from the form data is obtained by −
- user = request.form['nm']

- It is passed to **'/success'** URL as variable part. The browser displays a **welcome** message in the window.



- Change the method parameter to **'GET'** in **login.html** and open it again in the browser.

- The data received on server is by the **GET** method. The value of 'nm' parameter is now obtained by −

- User = request.args.get('nm')

- Here, **args** is dictionary object containing a list of pairs of form parameter and its corresponding value. The value corresponding to 'nm' parameter is passed on to '/success' URL as before.

# Part-2

# Flask – Templates

- It is possible to return the output of a function bound to a certain URL in the form of HTML.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def index():
   return '<html><body><h1>Hello World</h1></body></html>'

if __name__ == '__main__':
   app.run(debug = True)
```

# Flask – Templates

- One can take advantage of **Jinja2** template engine, on which Flask is based.

- Instead of returning hardcode HTML from the function, a HTML file can be rendered by the **render_template()** function

# Flask – Templates

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def index():
    return render_template('hello.html')

if __name__ == '__main__':
    app.run(debug = True)
```

# Flask – Templates

- Flask will try to find the HTML file in the templates folder, in the same folder in which this script is present.

- Application folder
  - Hello.py
  - templates
    - hello.html

- The term **‘web templating system’** refers to designing an HTML script in which the variable data can be inserted dynamically.

- Flask uses **jinja2** template engine

# Flask – Templates

```
<!doctype html>
<html>
   <body>

      <h1>Hello {{ name }}!</h1>

   </body>
</html>
```

save as **hello.html** in the templates folder.

Next, run the following script from Python shell.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<user>')
def hello_name(user):
   return render_template('hello.html', name = user)

if __name__ == '__main__':
   app.run(debug = True)
```

# Flask – Templates

enter URL as − **http://localhost:5000/hello/mvl**

# jinja2 template engine

- The **jinja2** template engine uses the following delimiters for escaping from HTML.

- {% ... %} for Statements
- {{ ... }} for Expressions to print to the template output
- {# ... #} for Comments not included in the template output
- # ... ## for Line Statements

The Python Script is as follows –

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<int:score>')
def hello_name(score):
   return render_template('hello.html', marks = score)

if __name__ == '__main__':
   app.run(debug = True)
```

HTML template script of **hello.html** is as follows –

```
<!doctype html>
<html>
   <body>
      {% if marks>50 %}
         <h1> Your result is pass!</h1>
      {% else %}
         <h1>Your result is fail</h1>
      {% endif %}
   </body>
</html>
```

# Flask – Templates

- The Python loop constructs can also be employed inside the template.

Run the following code from Python shell.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/result')
def result():
   dict = {'phy':50,'che':60,'maths':70}
   return render_template('result.html', result = dict)

if __name__ == '__main__':
   app.run(debug = True)
```

# Flask – Templates

Save the following HTML script as **result.html** in the templates folder.

```
<!doctype html>
<html>
   <body>
      <table border = 1>
         {% for key, value in result.items() %}
            <tr>
               <th> {{ key }} </th>
               <td> {{ value }} </td>
            </tr>
         {% endfor %}
      </table>
   </body>
</html>
```

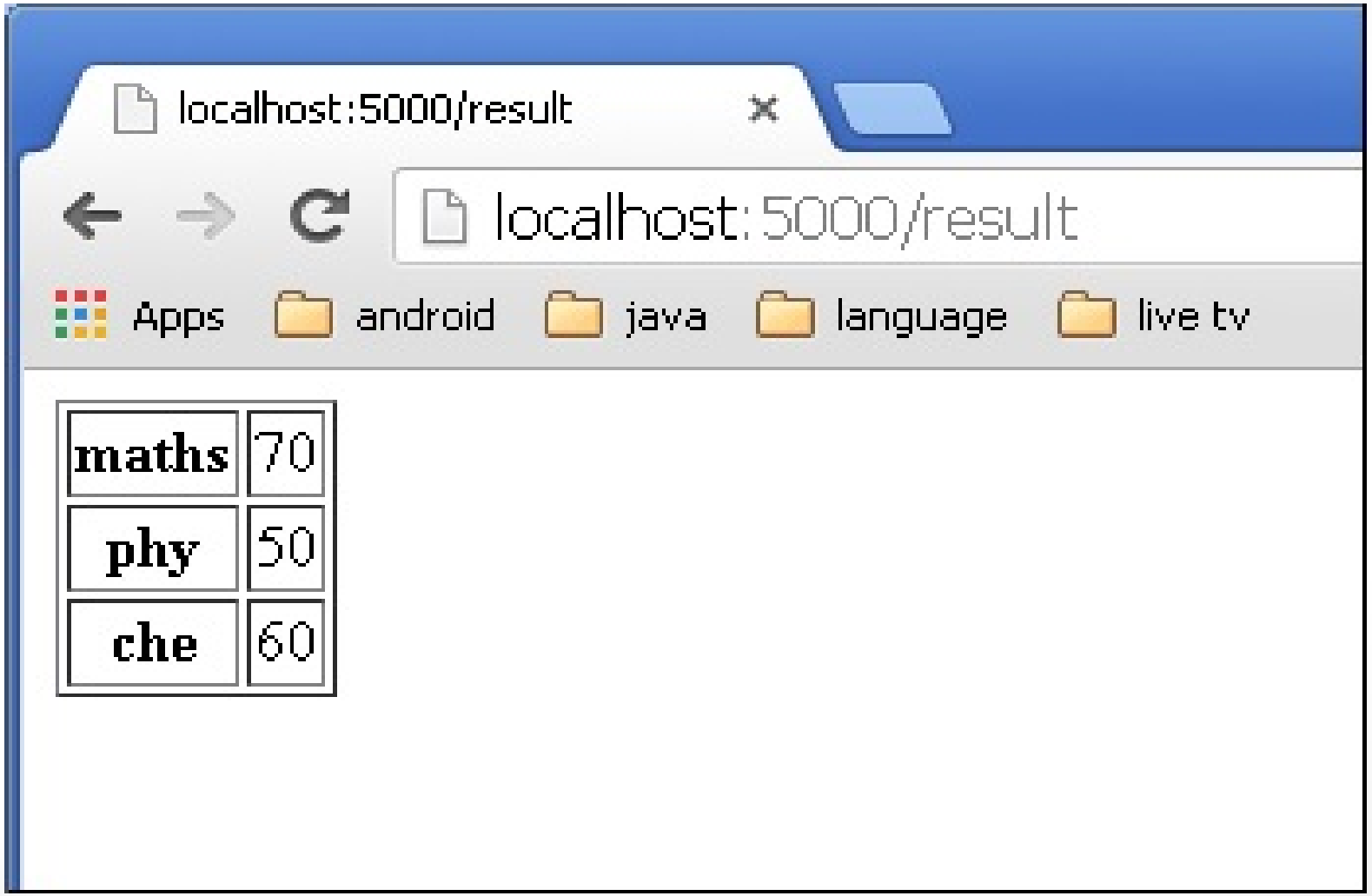
# Flask – Templates

- The Template part of **result.html** employs a **for loop** to render key and value pairs of dictionary object **result{}** as cells of an HTML table.

- Here, again the Python statements corresponding to the **For** loop are enclosed in {%..%} whereas, the expressions **key and value** are put inside **{{ }}**.

- After the development starts running, open **http://localhost:5000/result** in the browser to get the following output.

# Result


localhost:5000/result
localhost:5000/result
Apps
android
java
language
live tv
maths 70
phy 50
che 60

# Flask – Static Files

- A web application often requires a static file such as a **javascript** file or a **CSS** file supporting the display of a web page.
- web server is configured to serve them for you, but during the development, these files are served from *static* folder in your package
- It will be available at ***/static*** on the application.
- A special endpoint ‘static’ is used to generate URL for static files.

# Flask – Static Files

- In the following example, a **javascript** function defined in **hello.js** is called on **OnClick** event of HTML button in **index.html**, which is rendered on **'/'** URL of the Flask application.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/")
def index():
   return render_template("index.html")

if __name__ == '__main__':
   app.run(debug = True)
```

The HTML script of **index.html** is given below.

```
<html>
   <head>
      <script type = "text/javascript"
         src = "{{ url_for('static', filename = 'hello.js') }}" ></script>
   </head>

   <body>
      <input type = "button" onclick = "sayHello()" value = "Say Hello" />
   </body>
</html>
```

**hello.js** contains **sayHello()** function.

```
function sayHello() {
   alert("Hello World")
}
```

# Flask – Request Object

- The data from a client's web page is sent to the server as a global request object.
- In order to process the request data, it should be imported from the Flask module.

# Flask – Request Object

- Important attributes of request object are listed below −
  - **Form** − It is a dictionary object containing key and value pairs of form parameters and their values.
  - **args** − parsed contents of query string which is part of URL after question mark (?).
  - **Cookies** − dictionary object holding Cookie names and values.
  - **files** − data pertaining to uploaded file.
  - **method** − current request method.

Given below is the Python code of application –

```
from flask import Flask, render_template, request
app = Flask(__name__)

@app.route('/')
def student():
   return render_template('student.html')

@app.route('/result',methods = ['POST', 'GET'])
def result():
   if request.method == 'POST':
      result = request.form
      return render_template("result.html",result = result)

if __name__ == '__main__':
   app.run(debug = True)
```

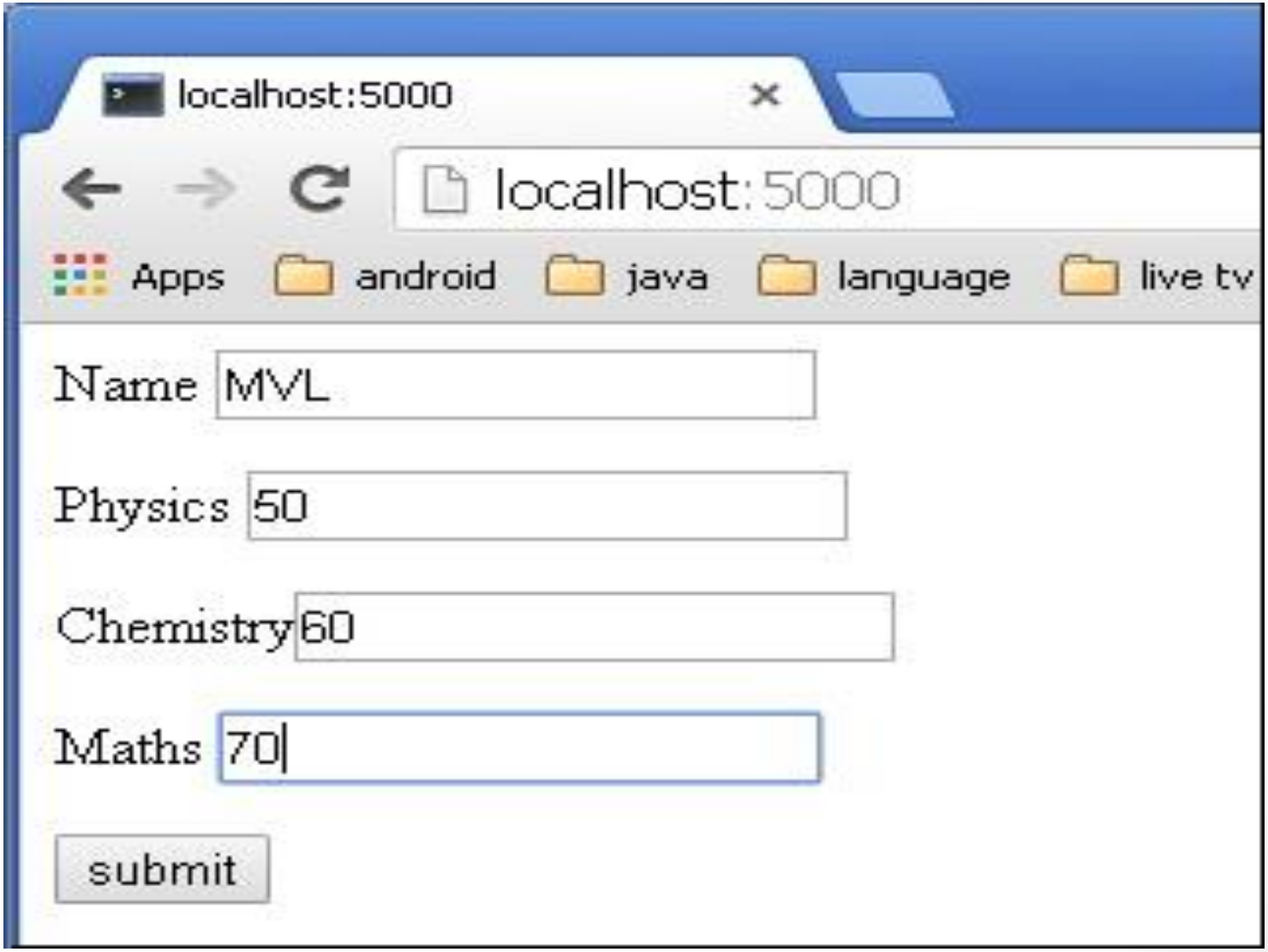
Given below is the HTML script of **student.html**.

```
<html>
   <body>
      <form action = "http://localhost:5000/result" method = "POST">
         <p>Name <input type = "text" name = "Name" /></p>
         <p>Physics <input type = "text" name = "Physics" /></p>
         <p>Chemistry <input type = "text" name = "chemistry" /></p>
         <p>Maths <input type ="text" name = "Mathematics" /></p>
         <p><input type = "submit" value = "submit" /></p>
      </form>
   </body>
</html>
```

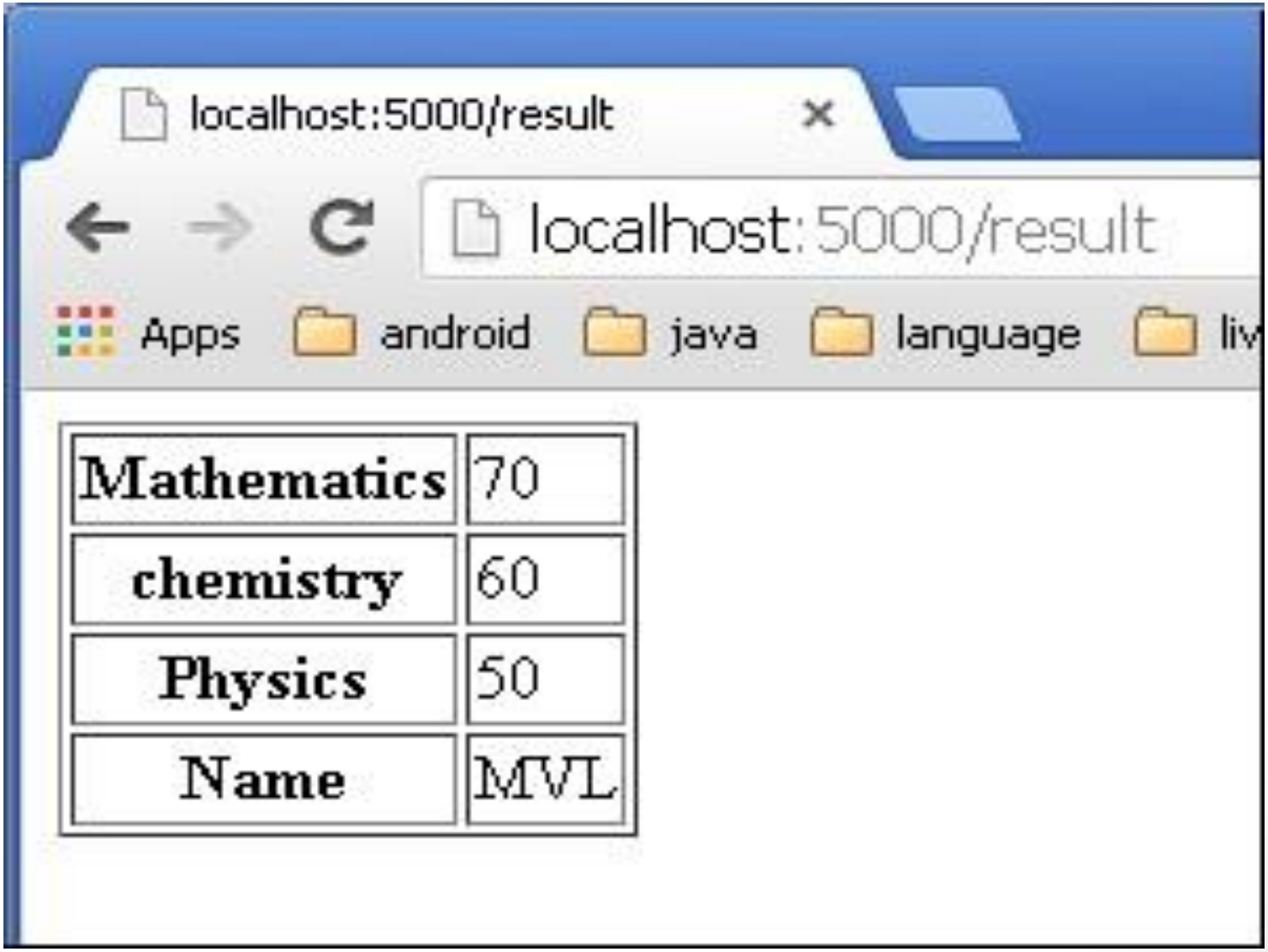
Code of template **(result.html)** is given below –

```
<!doctype html>
<html>
   <body>
      <table border = 1>
         {% for key, value in result.items() %}
            <tr>
               <th> {{ key }} </th>
               <td> {{ value }} </td>
            </tr>
         {% endfor %}
      </table>
   </body>
</html>
```

- Run the Python script and enter the URL **http://localhost:5000/** in the browser.



- When the **Submit** button is clicked, form data is rendered on **result.html** in the form of HTML table.



# PART-3

# Flask – Cookies

- A cookie is stored on a client's computer in the form of a text file.
- Its purpose is to remember and track data pertaining to a client's usage for better visitor experience and site statistics.
- A **Request object** contains a cookie's attribute.
- It is a dictionary object of all the cookie variables and their corresponding values, a client has transmitted. In addition to it, a cookie also stores its expiry time, path and domain name of the site.

# Flask – Cookies

- In Flask, cookies are set on response object.
- Use **make_response()** function to get response object from return value of a view function.
- After that, use the **set_cookie()** function of response object to store a cookie.
- Reading back a cookie is easy.
- The **get()** method of **request.cookies** attribute is used to read a cookie.

# Flask – Cookies

In the following Flask application, a simple form opens up as you visit '/' URL.

```
@app.route('/')
def index():
   return render_template('index.html')
```

This HTML page contains one text input.

```
<html>
   <body>
      <form action = "/setcookie" method = "POST">
         <p><h3>Enter userID</h3></p>
         <p><input type = 'text' name = 'nm'/></p>
         <p><input type = 'submit' value = 'Login'/></p>
      </form>
   </body>
</html>
```

- The Form is posted to '**/setcookie**' URL. The associated view function sets a Cookie name **userID** and renders another page.

```
@app.route('/setcookie', methods = ['POST', 'GET'])
def setcookie():
   if request.method == 'POST':
   user = request.form['nm']

   resp = make_response(render_template('readcookie.html'))
   resp.set_cookie('userID', user)

   return resp
```

- **‘readcookie.html’** contains a hyperlink to another view function **getcookie()**, which reads back and displays the cookie value in browser.

```
@app.route('/getcookie')
def getcookie():
   name = request.cookies.get('userID')
   return '<h1>welcome '+name+'</h1>'
```

- Run the application and visit **http://localhost:5000/**

- The result of setting a cookie is displayed like this −

- The output of read back cookie is shown below.

# Flask – Sessions

- Like Cookie, Session data is stored on client.
- Session is the time interval when a client logs into a server and logs out of it.
- The data, which is needed to be held across this session, is stored in the client browser.
- A session with each client is assigned a **Session ID**.

# Flask – Sessions

- The Session data is stored on top of cookies and the server signs them cryptographically.
- For this encryption, a Flask application needs a defined **SECRET_KEY**.
- Session object is also a dictionary object containing key-value pairs of session variables and associated values.

- URL ‘/’ simply prompts user to log in, as session variable ‘**username**’ is not set.

```
@app.route('/')
def index():
   if 'username' in session:
      username = session['username']
         return 'Logged in as ' + username + '<br>' + \
         "<b><a href = '/logout'>click here to log out</a></b>"
   return "You are not logged in <br><a href = '/login'></b>" + \
      "click here to log in</b></a>"
```

```
@app.route('/login', methods = ['GET', 'POST'])
def login():
    if request.method == 'POST':
        session['username'] = request.form['username']
        return redirect(url_for('index'))
    return '''

    <form action = "" method = "post">
        <p><input type = text name = username/></p>
        <p<<input type = submit value = Login/></p>
    </form>

    '''
```

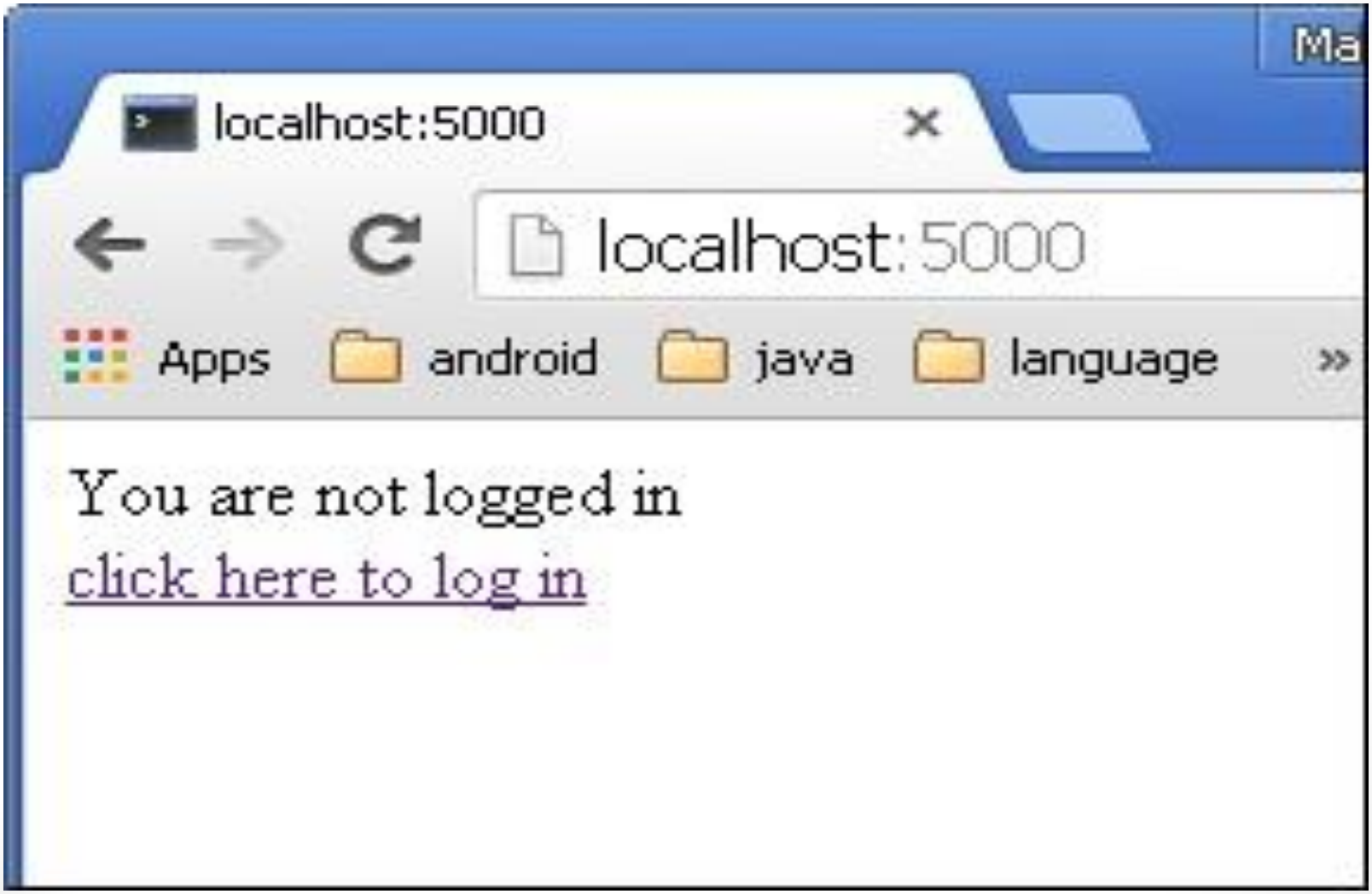
- The application also contains a **logout()** view function, which pops out **'username'** session variable. Hence, **'/'** URL again shows the opening page.

```
@app.route('/logout')
def logout():
   # remove the username from the session if it is there
   session.pop('username', None)
   return redirect(url_for('index'))
```

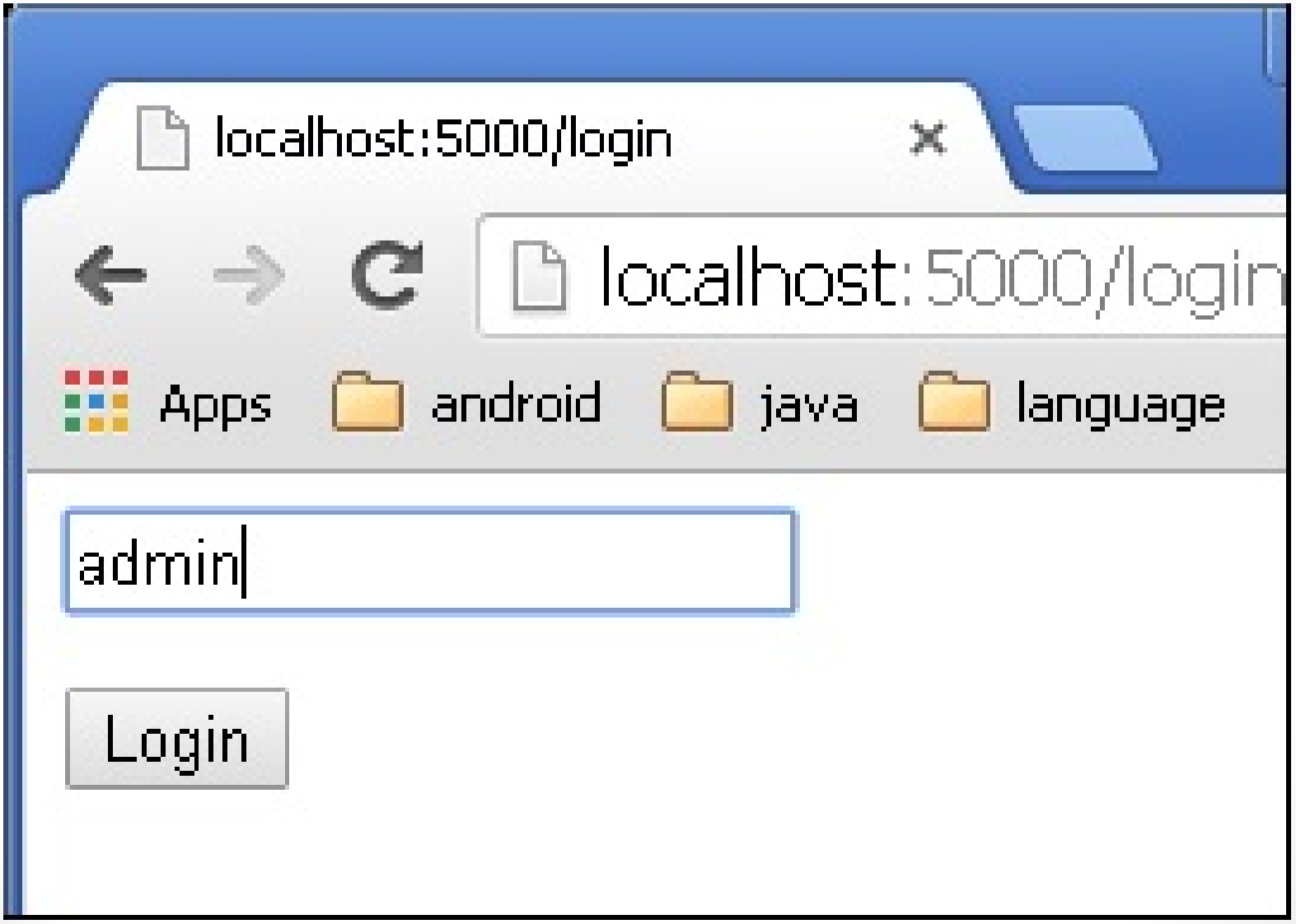
Run the application and visit the homepage. (Ensure to set **secret_key** of the application)

```
from flask import Flask, session, redirect, url_for, escape, request
app = Flask(__name__)
app.secret_key = 'any random string'
```

The output will be displayed as shown below. Click the link "click here to log in"



The link will be directed to another screen. Type ‘admin’.



The screen will show you the message,

**‘Logged in as admin’**

# Flask – File Uploading

- It needs an HTML form with its enctype attribute set to ‘multipart/form-data’, posting the file to a URL.
- The URL handler fetches file from **request.files[]** object and saves it to the desired location.
- Each uploaded file is first saved in a temporary location on the server, before it is actually saved to its ultimate location.
- Name of destination file can be hard-coded or can be obtained from filename property of **request.files[file]** object.

# Flask – File Uploading

- However, it is recommended to obtain a secure version of it using the **secure_filename()** function.

- The following code has **'/upload'** URL rule that displays **'upload.html'** from the templates folder, and **'/upload-file'** URL rule that calls **uploader()** function handling upload process.

# Flask – File Uploading

Following is the Python code of Flask application.

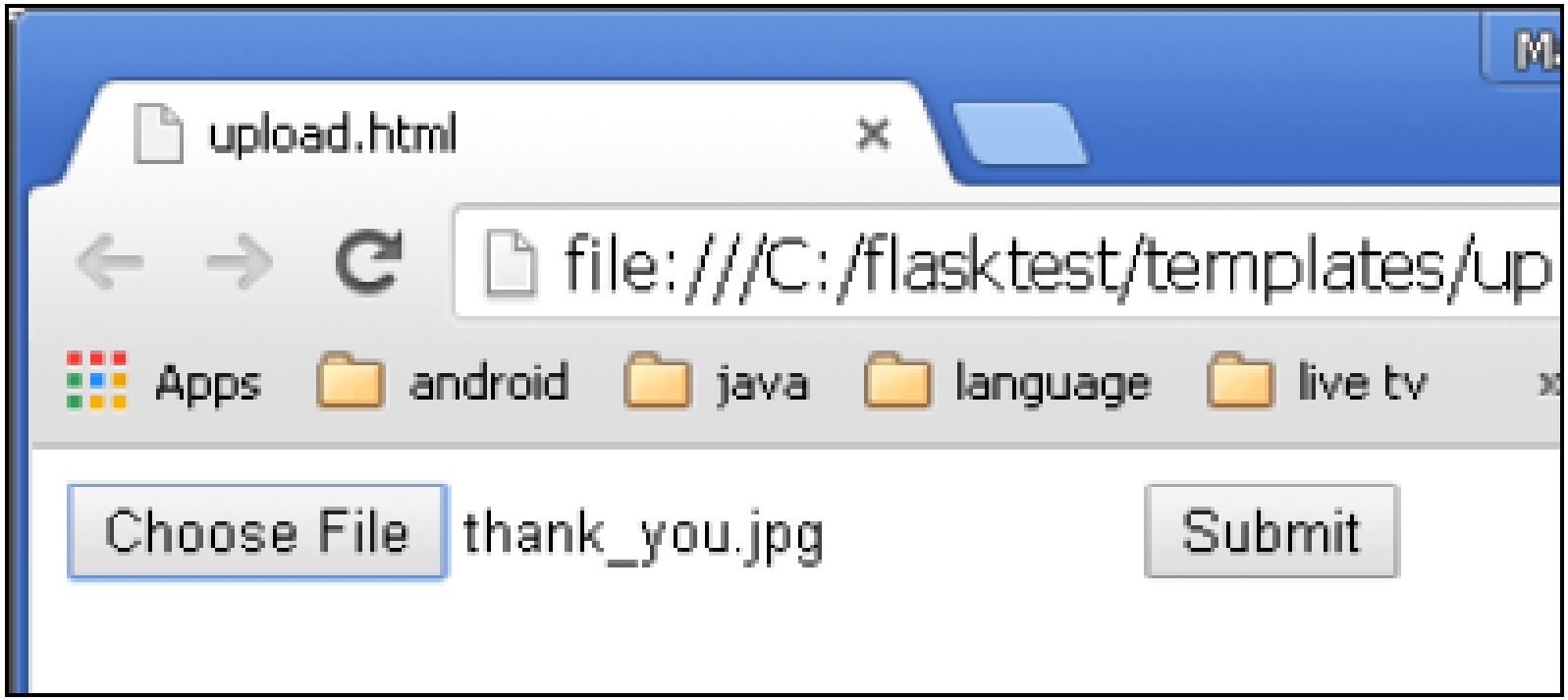
```
from flask import Flask, render_template, request
from werkzeug import secure_filename
app = Flask(__name__)

@app.route('/upload')
def upload_file():
   return render_template('upload.html')

@app.route('/uploader', methods = ['GET', 'POST'])
def upload_file():
   if request.method == 'POST':
      f = request.files['file']
      f.save(secure_filename(f.filename))
      return 'file uploaded successfully'

if __name__ == '__main__':
   app.run(debug = True)
```

# Flask – File Uploading

'**upload.html**' has a file chooser button and a submit button.

```
<html>
   <body>
      <form action = "http://localhost:5000/uploader" method = "POST"
         enctype = "multipart/form-data">
         <input type = "file" name = "file" />
         <input type = "submit"/>
      </form>
   </body>
</html>
```

# OUTPUT:




Click **Submit** after choosing file. Form's post method invokes **'/upload_file'** URL. The underlying function **uploader()** does the save operation.

# Thank for your attention